



TASK

Django – eCommerce Application Part 2

Visit our website

Introduction

WELCOME TO THE DJANGO E-COMMERCE APPLICATION TASK PART 2!

Now that we have built our eCommerce site, a secure web application with authentication and authorisation, it is time to expand our application to support an API. We will use our knowledge of authentication and authorisation to build a secure web API for our users. In this lesson, you will practise working with both GET and POST requests. The Django REST framework examples demonstrate how to create and add data using POST requests, while the JSONPlaceholder example demonstrates how an application can retrieve and display data from an external API using GET requests. Together, these activities provide practical experience in both building and consuming APIs.

APIS

Many applications communicate with other software services through APIs. For example, a weather application may retrieve forecast data from a weather service, an online store may communicate with a payment service, or a web application may retrieve information from another data provider. Web services can create APIs that allow other applications to retrieve, create, update, or delete data.

As a developer, you may therefore work with APIs in two different ways. You may build an API that allows other applications to interact with your own service, or you may consume an API provided by another service. In this task, you will practise both approaches.

In this task, we will explore how to create our own web API for an eCommerce service and how to consume data from another REST API. For the external API example, we will use **JSONPlaceholder**, a free fake REST API designed for learning, testing, and prototyping. This allows us to practise API integration without requiring API keys, developer accounts, or access approval. We will also learn how Postman can be used to test API requests and responses.

Web APIs

API is an acronym for “application programming interface” and it’s a set of rules that enable software applications to communicate with each other. Web APIs are APIs that allow applications to communicate over the Internet.

Web APIs use standard communication protocols such as HTTP and HTTPS to allow communication over the Internet. Clients can make a request to the API by sending an HTTP request. The API will then process the request and send back a response. This is similar to how our web service already works, but instead of returning HTML, a web API will typically respond with data in XML (Extensible Markup Language) or JSON (JavaScript Object Notation) format. XML defines a set of rules for encoding documents in a format that is both human- and machine-readable, and JSON is an open standard file format that uses human-readable text to transmit data objects consisting of attribute-value pairs and array data types.

When building a web API, it's wise to follow some sort of structure or architecture. Using a known standard makes it easier for other developers to use and integrate your API with their projects. It can also assist in preventing issues that can occur later by enabling scalability, flexibility, and independence. We will now take a look at the REST architecture and how we can use it to build a web API for our eCommerce service.

RESTful architecture

In the first task of this two-part task, we asked you to research RESTful APIs. Let's recap some of what you may have learned. REST (representational state transfer) is a software architecture or guideline, invented by Roy Fielding in 2000, for building APIs. The architecture comprises principles and constraints to achieve better performance, scalability, and modifiability.

The main feature of REST is **statelessness**. This means that each request will be treated as a new request and servers do not save client data between requests. Each request has to contain all the information, such as authentication data, a **GET/POST/PUT/DELETE** command, JSON, XML, etc., needed to understand and process the request. Without all the required information the API will not be able to successfully process the request and will ultimately fail to return the desired resource.

Here are some other features and constraints of REST:

Client-server architecture: REST uses the client-server architecture where the client and server communicate over a uniform interface using HTTP.

Uniform interface: RESTful systems have a uniform and consistent interface, which allows simplicity and scalability. We can create a uniform REST interface by following these four constraints:

- **Identification of resources:** Resources are identified with URIs (Uniform Resource Identifiers).
- **Manipulation of resources through representations:** Clients manipulate resources through the representation provided by the server. Representations should provide all information to modify or delete the resource or fetch additional data.
- **Hypermedia as the engine of application state (HATEOAS):** The client doesn't need prior knowledge of the structure of the application's APIs but can navigate and interact with the application based on the links provided by the server.

Cacheable: When a user makes the same call twice the data is retrieved from the cache instead of making a new API call.

Layered: The client does not know whether it is communicating with an end server or an intermediary such as a gateway.

REPRESENTATIONAL FORMATS

JSON

JSON (JavaScript Object Notation) is a storing and exchanging format that is very popular with web applications. While JSON has "JavaScript" in its name, it's important to note that **the format is language-independent** and only the notation was borrowed from JavaScript.

JSON is human-readable and easy to understand, and it uses key-value pairs similar to a Python dictionary. Values can be of any data type, including string, boolean, integer, float, etc. JSON is also very popular as it can be used in many programming languages, including Python, Java, C#, PHP, and many more.

Here is an example of the JSON format:

```
{
  "name": "Peter Jackson",
  "age": 25,
  "gender": "male",
  "hobbies": ["reading", "gaming", "coding"]
}
```



Take Note

To learn more about JSON you can have a look at the [official website](#).

XML

Another format that is also popular to use when communicating data structures is XML (Extensible Markup Language). Just like JSON, XML is platform-independent and human-readable. XML uses tags to mark up elements, and we can create these tags using angle brackets, "<" and ">", as with HTML. Each tag has an opening and closing component, where the closing tag will have a "/" before the element name in the tag (once again, as with HTML).

Here is an example of a basic element:

```
<person>
  <name>Peter Jackson</name>
  <age>25</age>
</person>
```

We can add attributes to elements as well to provide additional information:

```
<book title="The Catcher in the Rye" author="J.D. Salinger"/>
```

XML documents have a hierarchical structure, which means elements can contain other elements. Here is an example of this:

```
<library>
  <book>
    <title>1984</title>
    <author>George Orwell</author>
  </book>
  <book>
    <title>Brave New World</title>
    <author>Aldous Huxley</author>
  </book>
</library>
```

XML's simplicity and flexibility make it suitable for a variety of applications when exchanging and storing information.

MAKING AN API

Let's see how we can build an API for our web app using the RESTful architecture principles.

Basic API request

We can set up a basic API response to see how they are similar to and differ from the regular HTTP responses you might already be familiar with.

```
def basic_api_response(request):  
    if request.method == "GET":  
        data = serializers.serialize('json', Store.objects.all())  
        return JsonResponse(data=data, safe=False)
```

We first make sure that the user has made a **GET** request. We then run the data from our Store models through a serialiser to serialise it to JSON format. Once we have our data in JSON we can return a [JsonResponse](#) instead of an **HttpResponse**. The data from the response is set to the data produced by our serialiser, and we set the **safe** parameter to **False** to make sure any object instance is allowed to be serialised; without this, we can only serialise dictionary objects.

Output:

```
[{"model": "eCommerce.store", "pk": 1, "fields": {"vendor": 1, "name": "BaconShop", "description": "We sell Bacon!"}}, {"model": "eCommerce.store", "pk": 2, "fields": {"vendor": 1, "name": "Sock & Sandals", "description": "We sell sock! We sell sandals! Come check them out!"}}
```



Take Note

Serialisation in Django is where we translate our objects into different formats. To learn more about this, read the [Django documentation](#).

Using the Django REST framework

Unfortunately, the response from the method illustrated above does not produce a very user-friendly resource. When people use our API, we want the experience to be easy and efficient. To achieve this, we can use the Django REST framework to make the serialisation of our data a bit easier. You can install the Django Rest framework using the following command:

```
pip install djangorestframework
```

Sometimes you need a class containing a few attributes that would only be useful to one other class. We can call these classes “helper classes”. These helper classes also have access to the private attributes of the outer class. To create a helper class, we simply nest a class definition within the class we would like to attach the helper class to.

DRF serializers should be in a separate file, **serializers.py**

In `yourApp/serializers.py`

```
from rest_framework import serializers
from .models import Store
class StoreSerializer(serializers.ModelSerializer):
    class Meta:
        model = Store
        fields = ['vendor', 'name', 'description']
```

Here we created a class called **StoreSerializer** (note that Django uses the American spelling for the command, i.e., a “z” rather than an “s”, and so we’ve reflected this in the class name as well; this is not mandatory, but simply provides consistency with the spelling of the command). **StoreSerializer** will be used to serialise the data from our **Store** model. We set the superclass to **serializers.ModelSerializer**, then immediately create a new class inside our **StoreSerializer** class called **Meta**. This class will reference the model we are looking to serialise and we can also set the fields that should be added during serialisation.

Now let's update our previous view to use our new serialiser:

In our `yourApp/views.py`

```
from django.http import JsonResponse
from .models import Store
from .serializers import StoreSerializer

def view_stores(request):
    if request.method == "GET":
        stores = Stores.objects.all()

        serializer = StoreSerializer(stores, many=True)
        return JsonResponse(data=serializer.data, safe=False)
```

This time we use **StoreSerializer** instead of **serializers.serialize**. We can then make a query inside our serialiser to get all the values from our store table and serialise them. We also have to state in the serialiser that we are expecting multiple values in the query set by setting the **many** parameter to **True**. Finally, we return a **JsonResponse** object with our dataset to the data produced by our serialiser, and again we set the **safe** parameter to **False** to allow the serialisation of all object instances.

After using our new serialiser, our output looks much better:

```
[{"vendor": 1, "name": "BaconShop", "description": "We sell Bacon!"},
{"vendor": 1, "name": "Sock & Sandals", "description": "We sell sock! We sell sandals! Come check them out!"}]
```

When creating new views, remember to set their endpoints in the **urls.py** folder of your web app.

```
urlpatterns = [
    path('basic_response/', basic_api_response),
    path('get/stores', view_stores),
]
```

We can use the **api_view** decorator from the Django REST framework to make sure we receive **request** instances in our view, as well as to add context to our **Response** objects to allow content negotiation.

Let's see how we can add this to our `view_stores` view:

```
from rest_framework.decorators import api_view

@api_view(['GET'])
def view_stores(request):
    stores = Store.objects.all()
    serializer = StoreSerializer(stores, many=True)
    return Response(serializer.data)
```

We can also have our response return the data in XML format. But first, we need to install a package to assist us:

```
pip install djangorestframework-xml
```

The `djangorestframework-xml` package provides us with serialisers to format our data to XML. First, we have to add the following code to our `settings.py` file:

```
REST_FRAMEWORK = {
    'DEFAULT_RENDERER_CLASSES': (
        'rest_framework_xml.renderers.XMLRenderer',
    )
}
```

This will add the XML renderer to the rendered classes of our project.

We can then import the `render_classes` decorator and apply it to our view using the `XMLRenderer`:

```
from rest_framework.decorators import api_view, render_classes
from rest_framework_xml.renderers import XMLRenderer
from rest_framework.response import Response

@api_view(['GET'])
@render_classes((XMLRenderer,))
def view_stores(request):
    serializer = StoreSerializer(Store.objects.all(), many=True)
    return Response(serializer.data)
```

The `renderer_classes` decorator forces this view to render only XML, regardless of any global settings. You will also notice that we return a **Response** object instead of a **JsonResponse** object. The DRF (djangorestframework) will automatically serialise the response using the defined renderer (in this case, XML)

In addition to enabling users to view data using our API, we can allow them to add data, such as creating and editing stores. We can set up the following view for a user to add a store:

```
from rest_framework import status

@api_view(['POST'])
def add_store(request):

    serializer = StoreSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

We first check to see if the request coming in is a **POST** request. We then grab the data from our **request** object and run it through our **StoreSerializer**. Then we test to see if the data from the request is valid and in the correct format. If it's valid, we save the data and return it to the user with a success response code. Otherwise, we return the error messages with a bad request response code. We can return response codes to users by importing the **status** module from **rest_framework**.

When allowing users to view and especially add data to and from our database, we need to perform some authentication. Let's look at how to achieve this:

```
@api_view(['POST'])
@authentication_classes([BasicAuthentication])
@permission_classes([IsAuthenticated])
def add_store(request):
    vendor_id = request.data.get('vendor')
    if vendor_id is None:
        return Response({'error': 'Vendor field is required.'},
            status=status.HTTP_400_BAD_REQUEST)
    if int(vendor_id) != request.user.id:
        return Response({'error': 'User ID and vendor ID do not match.'},
            status=status.HTTP_403_FORBIDDEN)

    serializer = StoreSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

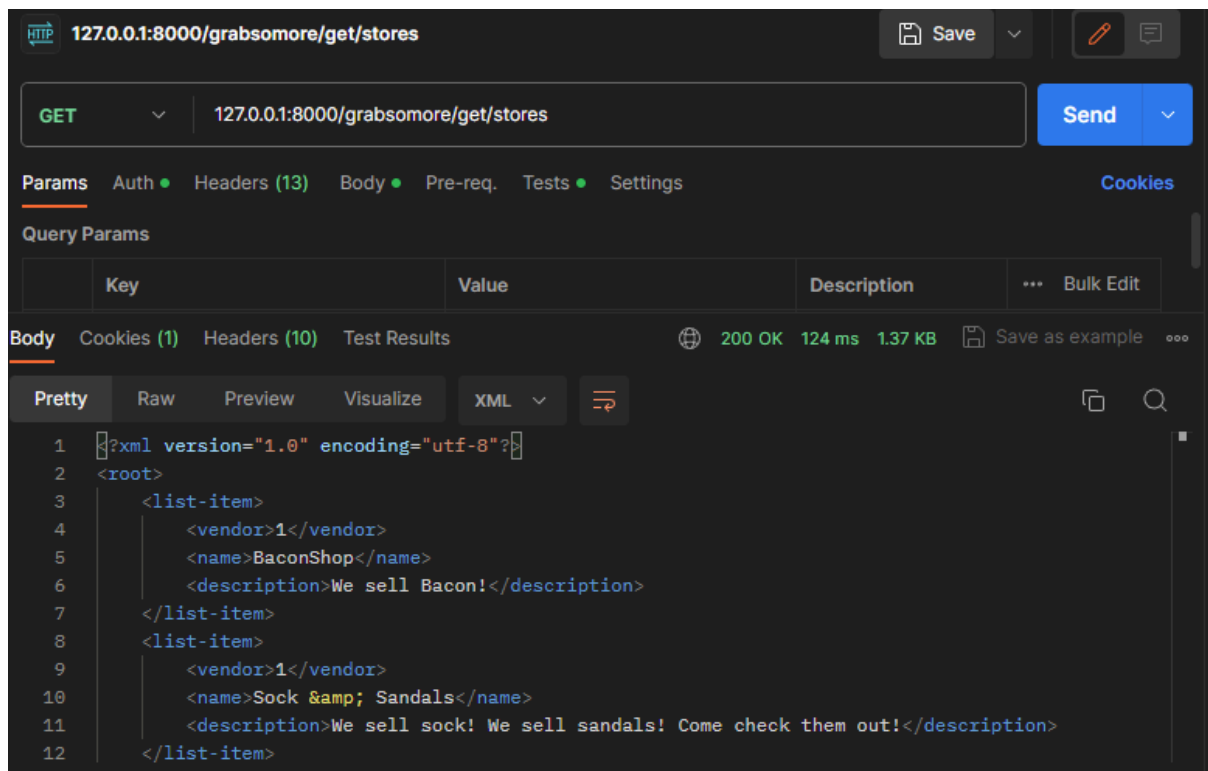
We have now added two more decorators to our function. The first, **authentication_classes**, allows us to use a specific authentication method. In this case, we will use **BasicAuthentication**, which requires a valid username and password to allow access to the view. We can then determine whether the authentication was successful by using the **permission_classes** decorator, adding the **IsAuthenticated** permission class to determine whether the user has been authenticated. A bonus of **BasicAuthentication** is that it attaches the user to the request. This allows us to determine whether the user is trying to add a new store to their own profile.

POSTMAN

To test our API, it would be quite inefficient to use a browser or to build a new application to make API requests. We can use an application called **Postman** to help us make HTTP requests very easily and attach all the necessary data with it.

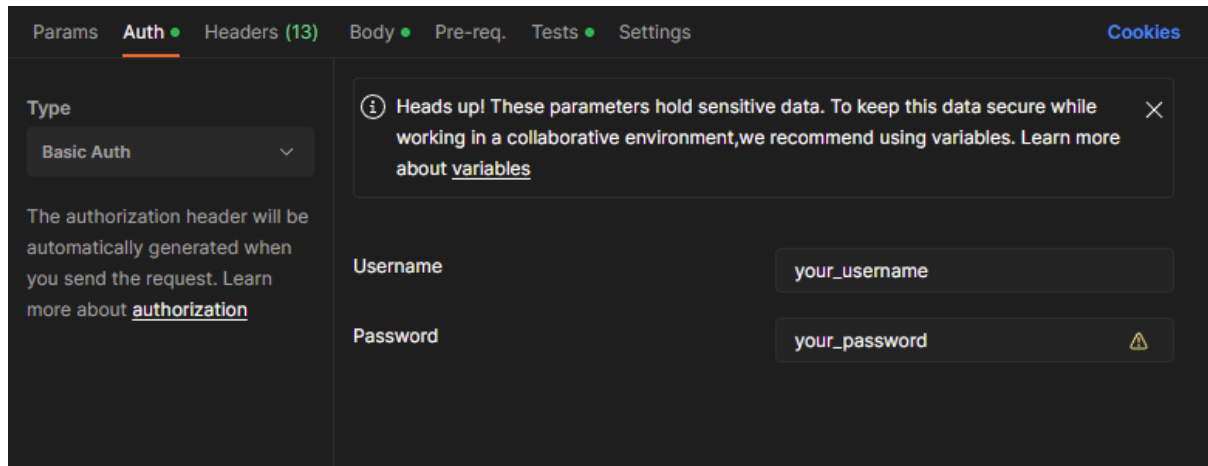
In Postman we can add the endpoints to our API where it says “Enter URL or paste text”. We can then set the request method and add the required data.

Let’s see what Postman will look like if we make a call to our **get/stores** endpoint:



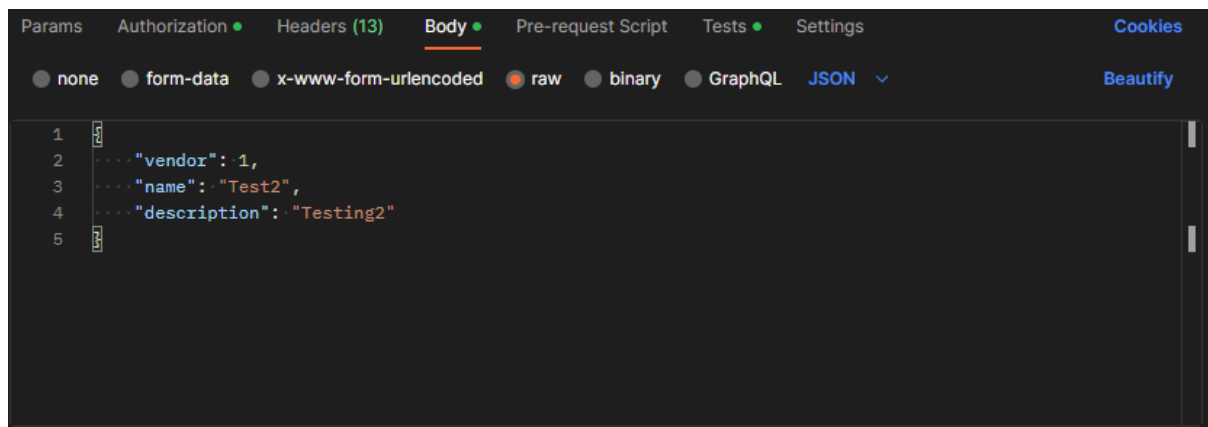
We make a request to the **get/stores** endpoint and set our request method to **GET**, as you can see to the left of the endpoint URI. When we hit send, we get the response you can see in the “Body” at the bottom of the image. This time the API returns the resource in XML form.

When making a request to an endpoint that requires authentication, you can navigate to the authentication (“Auth”) tab below the URL bar. Once on the authentication tab, you can select the type of authentication. We used “Basic Auth” for our **post/store** endpoint.



The screenshot shows the 'Auth' tab in a REST client. The 'Type' dropdown is set to 'Basic Auth'. A warning message states: 'Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about [variables](#)'. Below this, the 'Username' field contains 'your_username' and the 'Password' field contains 'your_password' with a warning icon.

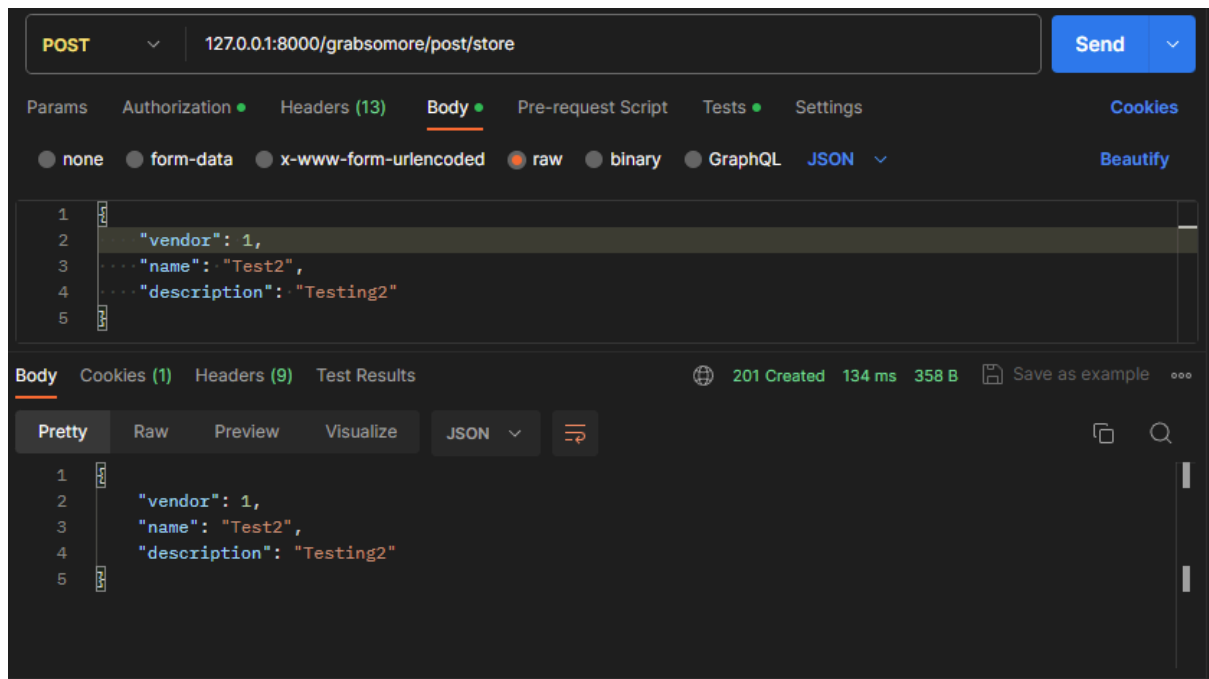
The **post/store** endpoint also requires the information of a new store in order to add it, and we can add this data to the body of our request. Navigate to the “Body” tab, select the data as “raw”, and set the type to “JSON”. Then create a store object in JSON format that represents a store in the same way it’s produced by our serialiser:



The screenshot shows the 'Body' tab in a REST client. The 'Body' tab is selected, and the 'Type' dropdown is set to 'JSON'. The data is entered in raw format as a JSON object:

```
{  "vendor": 1,  "name": "Test2",  "description": "Testing2"}
```

With all our data in place, we can now make a **POST** request to the **post/store** endpoint:



CONSUMING AN EXTERNAL REST API

Requests module

Now that we know how an API works, we can explore how a Django application can retrieve data from another REST API. We will use the Python **requests** module to send HTTP requests from our application.

The **requests** module allows Python applications to send requests such as GET, POST, PUT, PATCH, and DELETE and then work with the responses returned by a server. This makes it possible for one application to communicate with another service over HTTP.

For this example, we will use **JSONPlaceholder**. JSONPlaceholder provides fake REST API data that developers can use for learning and testing. It includes resources such as posts, comments, albums, photos, todos, and users. We will use its posts resource to practise retrieving JSON data and displaying it in our Django application.

Inside your virtual environment, run the following command in your console to install the **requests** library:


```
pip install requests
```

Now that we have installed the module, we can import it into a Python file:

```
import requests
```

With **requests** imported, we can use functions such as **get()**, **post()**, **put()**, and **delete()** to send HTTP requests.

For example, the following GET request retrieves posts from JSONPlaceholder:

```
response = requests.get(
    "https://jsonplaceholder.typicode.com/posts"
)
```

The response returned by the API contains information about the request and the data returned by the server.

To convert the JSON response into Python data, we can use the **json()** method:

```
posts = response.json()
```

The returned JSON objects can then be accessed using familiar Python structures such as dictionaries and lists.

JSONPlaceholder also supports filtering resources using query parameters. For example, the following request retrieves posts associated with user **1**:

```
response = requests.get(
    "https://jsonplaceholder.typicode.com/posts",
    params={"userId": 1}
)
```

The resulting request is equivalent to:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

This demonstrates how applications can send parameters to APIs to request specific data.

To learn more about the **requests** library, consult its official documentation.



Take Note

To learn more about **requests** you can have a look at the [official documentation](#).

Using the JSONPlaceholder API

For our external API example, we will use JSONPlaceholder to retrieve a collection of sample posts.

JSONPlaceholder is designed specifically for testing and educational purposes. The data it provides is not live user-generated content. Instead, it allows developers to practise making API requests and working with structured JSON responses in a predictable environment.

We will create our API call in a separate file. This keeps the code responsible for communicating with the external API separate from the rest of our Django application.

Follow along with the provided code

The completed code files used in the following example have been provided with your lesson resources. Open these files in your code editor and follow along as we work through each part of the integration:

```
functions/  
└─ jsonplaceholder.py  
  
views.py  
  
templates/  
└─ external_posts.html  
  
urls.py
```

We will begin with the `jsonplaceholder.py` file.

Create a **functions** folder in your Django application if one does not already exist, and create a file called **`jsonplaceholder.py`**.

In **`functions/jsonplaceholder.py`**:

```
import requests

def get_posts(user_id=None):
    url = "https://jsonplaceholder.typicode.com/posts"

    params = {}

    if user_id is not None:
        params["userId"] = user_id

    try:
        response = requests.get(
            url,
            params=params,
            timeout=10
        )

        response.raise_for_status()

        data = response.json()
        posts = []

        for item in data:
            posts.append({
                "id": item["id"],
                "title": item["title"],
                "body": item["body"],
                "user_id": item["userId"]
            })

        return posts

    except requests.RequestException:
        return []
```

Let's look at how this function works.

We begin by importing the **requests** module:

```
import requests
```

We then create a function called **get_posts()**. The function accepts an optional parameter called **user_id**.

```
def get_posts(user_id=None):
```

This means we can retrieve all available posts or request only posts associated with a particular sample user.

Next, we define the URL of the JSONPlaceholder posts endpoint:

```
url = "https://jsonplaceholder.typicode.com/posts"
```

We also create an empty dictionary called **params**:

```
params = {}
```

This dictionary will hold any query parameters that need to be included in the request.

If a **user_id** has been provided, we add it to the dictionary:

```
if user_id is not None:  
    params["userId"] = user_id
```

For example, if the function is called as follows:

```
get_posts(user_id=1)
```

the resulting request will include the following query parameter:

```
?userId=1
```

This allows the API to return only posts associated with sample user **1**.

We then send the GET request:

```
response = requests.get(  
    url,  
    params=params,  
    timeout=10  
)
```

The **url** tells the **requests** module which API endpoint to contact, while **params** adds any required query parameters.

The **timeout** value prevents the application from waiting indefinitely if the API does not respond.

Next, we use:

```
response.raise_for_status()
```

This checks whether the API returned a successful HTTP response. If the server returns an error response, such as a **404** or **500** status code, an exception will be raised.

Once a successful response has been received, we convert the JSON response into Python data:

```
data = response.json()
```

JSONPlaceholder returns a list of post objects. Each post contains information such as an ID, title, body, and user ID.

We then loop through the returned data and extract the fields needed by our application:

```
posts = []

for item in data:
    posts.append({
        "id": item["id"],
        "title": item["title"],
        "body": item["body"],
        "user_id": item["userId"]
    })
```

Finally, we return the completed list:

```
return posts
```

The request is placed inside a **try** and **except** block:

```
except requests.RequestException:
    return []
```

If a request-related error occurs, the function returns an empty list instead of allowing the entire application to fail.

Importing and Using the JSONPlaceholder Function

Now that we have created the helper function in `functions/jsonplaceholder.py`, the next step is to use it inside our Django application.

Open the supplied `views.py` file and locate the following code.

In `views.py`:

```
from django.shortcuts import render
from .functions.jsonplaceholder import get_posts

def external_posts(request):
    posts = get_posts()

    return render(
        request,
```

```
        "external_posts.html",  
        {"posts": posts}  
    )
```

We first import the `get_posts()` function:

```
from .functions.jsonplaceholder import get_posts
```

This allows our Django view to use the API helper function we created earlier.

We then create a view called `external_posts`:

```
def external_posts(request):
```

Inside the view, we call the helper function:

```
    posts = get_posts()
```

The function sends the request to JSONPlaceholder and returns the processed list of posts.

We then pass these posts into the Django template:

```
    return render(  
        request,  
        "external_posts.html",  
        {"posts": posts}  
    )
```

The `posts` key makes the returned data available inside the template.

The flow is therefore:

```
JSONPlaceholder  
    ↓  
get_posts()  
    ↓  
external_posts view
```

↓
`external_posts.html`

Filtering the Data Returned by the API

The helper function can also retrieve posts associated with a particular sample user.

For example:

```
posts = get_posts(user_id=1)
```

When this function is called, the following request is sent:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

The **userId=1** value is a query parameter. It tells JSONPlaceholder to return only posts associated with that sample user.

Query parameters are commonly used by APIs to filter, search, sort, or control the data returned by an endpoint.

For example:

```
def external_posts(request):  
    posts = get_posts(user_id=1)  
  
    return render(  
        request,  
        "external_posts.html",  
        {"posts": posts}  
    )
```

In this version of the view, only posts associated with sample user **1** will be passed into the template.

Creating the Template to Display Posts

Now that our view is passing the JSONPlaceholder data into the template, the next step is to display the posts on the page.

Open the supplied `external_posts.html` file.

In `templates/external_posts.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>External API Posts</title>
</head>
<body>
  <h1>Posts from JSONPlaceholder</h1>

  {% if posts %}
    <ul>
      {% for post in posts %}
        <li>
          <h2>{{ post.title }}</h2>

          <p>
            {{ post.body }}
          </p>

          <p>
            Sample user ID: {{ post.user_id }}
          </p>
        </li>
      {% endfor %}
    </ul>
  {% else %}
    <p>Posts could not be loaded at this time.</p>
  {% endif %}
</body>
</html>
```

The template first checks whether posts are available:

```
{% if posts %}
```

If posts have been returned, we loop through them:

```
{% for post in posts %}
```

For each post, we display its title:

```
{{ post.title }}
```

We then display the body of the post:

```
{{ post.body }}
```

Finally, we display the sample user associated with the post:

```
{{ post.user_id }}
```

If the API request was unsuccessful and the helper function returned an empty list, the following fallback message is displayed:

```
<p>Posts could not be loaded at this time.</p>
```

This means the application can still display a meaningful response even when the external API cannot be reached.

Adding the URL Configuration

The final step is to create a URL that allows users to access the page.

Open the supplied `urls.py` file and locate the route for the external posts view.

In `urls.py`:

```
from django.urls import path
from . import views
```

```
urlpatterns = [  
    path(  
        "external-posts/",  
        views.external_posts,  
        name="external_posts"  
    ),  
]
```

The following path:

```
"external-posts/"
```

is connected to:

```
views.external_posts
```

When a user visits:

```
http://localhost:8000/external-posts/
```

Django calls the `external_posts` view.

The complete process now looks like this:

```
User visits /external-posts/  
    ↓  
urls.py directs the request  
    ↓  
external_posts view runs  
    ↓  
get_posts() sends a GET request  
    ↓  
JSONPlaceholder returns JSON  
    ↓  
The data is processed  
    ↓  
The view passes the posts to the template  
    ↓
```

```
external_posts.html displays the posts
```

You have now connected a Django application to an external REST API and displayed the returned data in the browser.



Take note

JSONPlaceholder is a fake REST API intended for learning, testing, and prototyping. The posts used in this example are sample data and do not represent live users or production content.

The same general process demonstrated here can be used when working with other REST APIs. An application sends an HTTP request to an endpoint, receives a response, processes the returned data, and uses that data within the application.

The supplied code files provide the complete example shown in this section. Open each file in your code editor as you work through the explanations and observe how the different parts of the integration connect.



Practical task 1

In this task, you will extend the functionality of your eCommerce project by creating a RESTful API that allows users to interact with your service.

Before adding this functionality to your project, plan how your API will work.

- Consider your existing models and decide how their data should be serialised and represented, for example in JSON or XML.
- Carefully plan your API endpoints, as these determine how users and other applications will access your resources. You can see here for a good set of **best practices**.
- Create a sequence diagram that shows the main interactions a user will have with your API.

Then complete the following:

1. Allow authenticated vendors to create new stores through your web API.
2. Allow authenticated vendors to add products to their own stores through your web API.
3. Allow vendors to retrieve reviews through the API.
4. Ensure that authentication and appropriate access controls are applied before allowing users to create or modify resources.
5. Allow both buyers and vendors to retrieve:
 - the stores associated with each vendor
 - the products available within each store
6. Test your API endpoints and confirm that authorised users can access the appropriate functionality and that unauthorised users are prevented from modifying protected resources.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Practical task 2

Now that you have created your own API for third-party users, let's incorporate an external REST API into your project. In this task, you will use JSONPlaceholder, a free fake REST API designed for learning and testing.

Before beginning this task, review the supplied `jsonplaceholder.py`, `views.py`, `external_posts.html`, and `urls.py` example files from the lesson. Use the example to guide the structure of your own implementation.

Follow these steps:

1. Use the JSONPlaceholder `/posts` endpoint to retrieve sample post data.
2. Create a helper function in a new file called `functions/jsonplaceholder.py`.
3. Your helper function should:
 - Send a GET request to the JSONPlaceholder `/posts` endpoint.
 - Accept an optional user ID and use it as a query parameter to filter the posts when provided.
 - Check whether the request was successful.
 - Parse the JSON response into a list of posts.
 - Extract the `id`, `title`, `body`, and `userId` values from each post.
 - Handle request errors so that your application does not crash if the API cannot be reached.
4. Import your helper function into `views.py` and create a new view called `external_posts` that calls the function and passes the returned posts into a template.
5. Create a template called `external_posts.html` that loops through the posts and displays the:
 - title
 - body
 - sample user ID
6. Include an appropriate fallback message in the template that is displayed if the posts cannot be retrieved.
7. Add a URL pattern in `urls.py` that maps the `/external-posts/` path to your new `external_posts` view.
8. Test your integration by running the Django development server and visiting the `/external-posts/` endpoint. Confirm that the sample posts returned by JSONPlaceholder are displayed correctly.

9. Test the filtering functionality by retrieving posts associated with a specific sample user, such as user **1**. Confirm that only posts associated with the selected **userId** are returned and displayed correctly.

JSONPlaceholder provides sample data for development, learning, and testing. You are being assessed on your ability to consume an external REST API, process the returned JSON data, handle possible request errors, and use the data within a Django application.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
